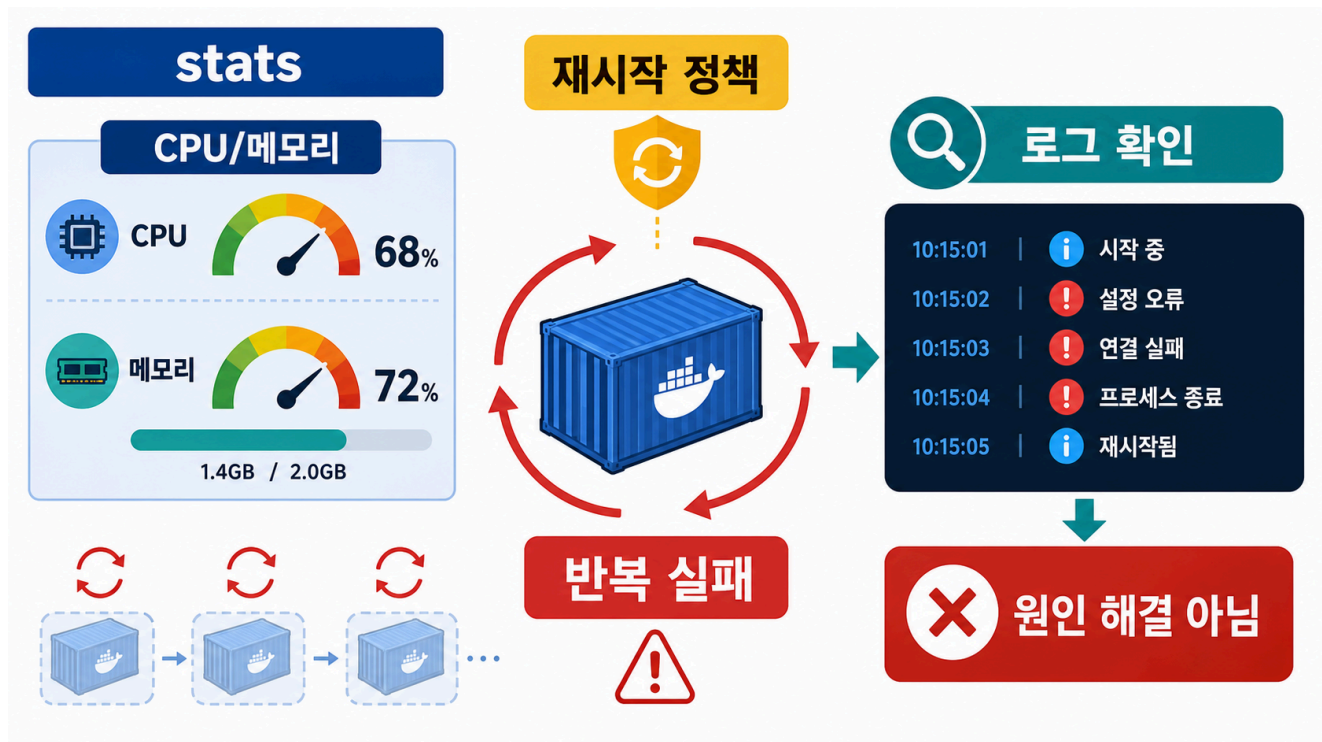


4교시: Stats, resource, restart policy



수업 목표

- `docker stats --no-stream`으로 resource 사용량을 확인한다.
- restart policy가 무엇을 해주고 무엇을 해주지 못하는지 설명한다.
- crash loop를 증상 완화와 원인 해결로 구분한다.

개념 설명

`docker stats`는 container의 CPU, memory, network, block I/O를 보여준다. Day 4에서는 성능 튜닝을 깊게 하지 않는다. 대신 실행 중인 container가 resource를 쓰고 있는지, 비정상적으로 반복 재시작되는지를 관찰하는 입구로 사용한다.

Restart policy는 container process가 죽었을 때 다시 시작할지 정하는 정책이다. 하지만 설정 누락, 잘못된 command, port 충돌 같은 원인을 고치지는 않는다. 반복 재시작은 장애를 숨길 수도 있으므로 logs와 함께 봐야 한다.

환경 설정이 잘못된 상태에서 restart policy를 붙이면 더 헛갈릴 수 있다. 예를 들어 required env가 없어서 바로 죽는 container에 `--restart on-failure`를 붙이면 container가 반복해서 재시작한다. 이때 해결책은 restart 횟수를 늘리는 것이 아니라 missing env를 고치는 것이다.

실습 명령

```
docker stats paperclip-day4-nginx --no-stream
docker inspect paperclip-day4-nginx --format 'before={{json
.HostConfig.RestartPolicy}}'
docker update --restart unless-stopped paperclip-day4-nginx
docker inspect paperclip-day4-nginx --format 'after={{json
.HostConfig.RestartPolicy}}'
```

Expected:

```
NAME                CPU %      MEM USAGE / LIMIT
before={"Name": "no", "MaximumRetryCount": 0}
after={"Name": "unless-stopped", "MaximumRetryCount": 0}
```

crash loop 맛보기

```
docker rm -f paperclip-day4-crash || true
docker run -d --name paperclip-day4-crash --restart on-failure:3 alpine:3.20
sh -c 'echo crash-now; exit 1'
sleep 3
docker ps -a --filter name=paperclip-day4-crash
docker logs paperclip-day4-crash
docker inspect paperclip-day4-crash --format 'RestartCount={{.RestartCount}}
Status={{.State.Status}} ExitCode={{.State.ExitCode}}'
```

Expected:

```
crash-now
RestartCount=3
ExitCode=1
```

config 실패와 restart 구분

```
docker rm -f paperclip-day4-restart-missing-env || true
docker run -d --name paperclip-day4-restart-missing-env --restart on-failure:2
postgres:16-alpine
sleep 3
docker inspect paperclip-day4-restart-missing-env --format 'RestartCount=
```

```
{{.RestartCount}} Status={{.State.Status}} ExitCode={{.State.ExitCode}}'
docker logs paperclip-day4-restart-missing-env --tail 20 || true
```

Expected:

```
RestartCount=2
POSTGRES_PASSWORD
```

해석: restart policy는 missing env를 해결하지 못한다. POSTGRES_PASSWORD를 주입해야 한다.

판단 기준

출력	해석
CPU %, MEM USAGE	resource 관찰 출발점
RestartCount 증가	process가 반복 실패
ExitCode=1	command가 실패 종료
logs에 같은 줄 반복	restart가 원인을 해결하지 못함
POSTGRES_PASSWORD 반복	config 누락을 restart로 가리고 있음

운영 판단

resource 수치가 높다고 곧바로 restart policy를 바꾸는 것은 아니다. 먼저 logs로 error를 보고, inspect로 restart count와 exit code를 확인하고, 필요하면 exec로 내부 상태를 본다. Day 4의 기준은 많이 재시작한다가 아니라 왜 재시작하는지 증거를 모은다.

왜 여기서 끝내면 약한가

docker stats --no-stream은 snapshot이다. 수업에서 한 번 보고 끝내면 "CPU가 몇 퍼센트였다" 정도만 남는다. 운영에서 더 중요한 질문은 보통 다음과 같다.

질문	docker stats만으로 부족한 이유	다음 단계
언제부터 CPU가 올라갔는가	과거 추세가 없다	Prometheus time series
어떤 container가 계속 증가하는가	순간값만 보고 놓칠 수 있다	Grafana panel 또는 Explore
spike 시점에 무슨 log가 있었는가	metrics에는 log line이 없다	Loki 또는 docker logs
재시작 직전에 resource가 어땠는가	이전 상태가 사라진다	metrics retention

그래서 Day 4 마지막 preview에서 Prometheus/Grafana/cAdvisor/Loki를 살짝 본다. 깊게 배우는 것이 아니라, stats가 metrics observability로 확장된다는 감각을 잡는 것이 목적이다.

다음 연결

다음 교시는 여러 failure를 한 번에 분류하고 복구 기준을 세운다.